

Aufgabe Klausurvorbereitung – AstroFreight Logistics

Kontext

Du arbeitest an der Backend-Software für „AstroFreight“, ein fiktives Logistikunternehmen für interplanetare Frachtflüge. Das System soll täglich eingehende Frachtmissionen speichern, filtern, nach Planeten gruppieren und statistisch auswerten.

Deine Aufgabe ist es, das System auszuprogrammieren und dabei fortgeschrittene Konzepte der Java-Programmierung (Stream API, Funktionale Interfaces, Optionals, Records und Collections) korrekt anzuwenden.

1) Das Datenmodell

Wir verwenden für die Frachtmissionen ein record namens *CargoMission*. Es besteht aus folgenden Eigenschaften:

- String missionId (Eindeutige ID der Mission, z.B. „M-001“)
- String planet (Zielplanet, z.B. „Mars“)
- double payloadKg (Gewicht der Fracht in Kilogramm)
- int urgency (Dringlichkeitsstufe von 1 bis 5, wobei 5 höchste Priorität ist)

2) Aufgaben für die Klasse FreightManager

Implementiere die folgenden Aufgaben in der bereitgestellten Klasse *FreightManager*.

Aufgabe 1: Datentypen

Damit **keine** Frachtmission versehentlich **doppelt** im System landet, verwenden wir eine geeignete Datenstruktur. *Welche Datenstruktur wird benötigt?*

- Deklariere in der Klasse eine Membervariable vom Typ `Set<CargoMission>` und initialisiere sie als `HashSet`.
- Implementiere die Methode `addMission(CargoMission mission)`, die eine Mission zum Set hinzufügt.
 - Füge ebenfalls ein Print Statement hinzu, welches die neue Größe des Sets in der Konsole ausgibt
- *Theorie-Frage (Beantworte dies in einem Kommentar im Code):* Warum funktioniert die Duplikat-Prüfung (also das Verhindern gleicher Elemente im `HashSet`) bei unserem Datenmodell out-of-the-box, ohne dass wir die Methoden `equals()` und `hashCode()` selbst überschreiben müssen?

Aufgabe 2: Stream API & Optional

Die Routenplaner möchten wissen, welche Mission für einen bestimmten Planeten am schwersten ist.

- Implementiere die Methode `CargoMission.getHeaviestMissionTo(String planet)`.
- Nutze die Java Stream API auf deinem Set:
 - Filtere nach dem übergebenen Planeten.
 - Finde anschließend das Element mit dem höchsten Gewicht. Du benötigst hierfür einen passenden Comparator.
- Überprüfe, ob ein Maximum gefunden werden konnte
 - Wenn ja, gebe es zurück
 - Wenn nein, gebe null zurück

Aufgabe 3: Fortgeschrittene Streams

Für die Buchhaltung soll eine Übersicht erstellt werden, wie viel Kilogramm Fracht im Durchschnitt pro Planet transportiert wird.

- Implementiere die Methode `getAveragePayloadPerPlanet()`. *Was ist der Return type?*
- Nutze die Stream API auf deinem Set.
 - Gruppieren die Missionen nach dem Zielplaneten
 - Berechne dabei gleichzeitig das durchschnittliche Gewicht für die jeweilige Gruppe.
 - Gib das Ergebnis zurück

Aufgabe 4.1: Funktionale Programmierung

Um den Code flexibel zu halten, definieren wir Logik in Form von Lambda-Ausdrücken, die wir in Variablen speichern. Erstelle in der Methode **generateUrgencyReport()** folgende Variablen:

- Einen Ausdruck, welcher prüft, ob die Dringlichkeit (urgency) **größer oder gleich 4** ist.
- Einen Ausdruck, die ein Missions-Objekt in folgende Alert-Struktur umwandelt:
„ACHTUNG: [missionId] zum [planet] mit [payloadKg]kg Fracht!“
- Einen Ausdruck, der einen Alert (string) wie folgt in der Konsole ausgibt:
„DASHBOARD-ANZEIGE: [alert]“

Aufgabe 4.2: Streams

- Nutze erneut die Stream API auf deinem Set in der Methode **generateUrgencyReport()**.
- Filtere den Stream mit deinem Lambda-Ausdruck
- Sortiere die gefilterten Missionen alphabetisch nach der missionId.
- Transformiere (map) die Elemente anschließend mit deinem Lambda-Ausdruck
- Führe auf jedem Element des Streams die Aktion aus, die du im letzten Lambda-Ausdruck definiert hast

3) Main-Klasse zum Testen

Die Main-Klasse wurde schon programmiert, sodass du das AstroFreight Logistics System ausführen und testen kannst. Entferne dazu die Kommentare am Anfang und am Ende der Klasse.